

DATA ANALYTICS

**FASE 3 | AULA 03 -
STACKS MODERNAS, DATA
WAREHOUSE, DATA LAKE E
LAKEHOUSE, DATA MESH E DATA
FABRIC**

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS	16
O QUE VOCÊ VIU NESTA AULA?	18
REFERÊNCIAS	19

EMANDA

O QUE VEM POR AÍ?

Nesta aula, você aprenderá sobre os principais conceitos de Data Warehouses, Data Lakehouses e Data Fabrics. Veremos como essas abordagens são utilizadas para gerenciar e analisar grandes volumes de dados, facilitando a tomada de decisões informadas nas organizações. Além disso, você terá a oportunidade de realizar exercícios práticos que consolidarão o seu aprendizado.



HANDS ON

Preparação do Ambiente Data Lake e sua evolução para um Data Lakehouse

Vejamos os pré-requisitos e ferramentas:

- AWS S3 (ou Azure Blob Storage, GCP Storage para Data Lake).
- Apache Spark ou PySpark (para ETL).
- Delta Lake (para habilitar o Data Lakehouse).
- Ferramenta de BI (ex.: Power BI ou Tableau) ou Jupyter Notebooks para consumo.

Confira os próximos passos na seção “**Saiba Mais**”.

SAIBA MAIS

Instalações locais: Docker (opcional), Python, bibliotecas (boto3, pyspark, delta-spark).

Dados de exemplo: fornecer arquivos CSV ou JSON.

Configuração Inicial: criar buckets no S3 com estrutura básica.

- raw-data/: dados brutos.
- processed-data/: dados processados.
- lakehouse-data/: dados em formato otimizado (Delta).

Construindo o Data Lake

Passo 1 - Ingestão de Dados:

Configurar scripts em Python para enviar arquivos CSV ou JSON ao bucket S3.

Exemplo de script (usando boto3):

```
import boto3

s3_client = boto3.client('s3')

def upload_to_s3(file_name, bucket, key):
    s3_client.upload_file(file_name, bucket, key)

upload_to_s3('dados.csv', 'meu-bucket', 'raw-data/dados.csv')
```

Demonstrar como verificar o upload no console do AWS S3.

Passo 2 - Processamento ETL:

- Usar PySpark para processar os dados no bucket S3.
- Script básico de transformação:

```
python

from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("DataLake").getOrCreate()

# Ler dados brutos do S3

raw_data = spark.read.csv("s3://meu-bucket/raw-data/dados.csv",
header=True, inferSchema=True)

# Transformação simples

processed_data = raw_data.filter(raw_data['coluna'] > 0)

# Salvar no bucket processed-data

processed_data.write.csv("s3://meu-bucket/processed-data/",
mode="overwrite", header=True)
```

Evoluindo para o Data Lakehouse

Passo 1 - Introdução ao Delta Lake:

- Explicar o conceito de ACID Transactions e versionamento.
- Configurar Delta Lake no PySpark:

```
from delta import *

builder = SparkSession.builder \

    .appName("Lakehouse") \

    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension") \

    .config("spark.sql.catalog.spark_catalog",
```

```
"org.apache.spark.sql.delta.catalog.DeltaCatalog")
```

```
spark = builder.getOrCreate()
```

Passo 2 - Criando a Tabela Delta

Converter os dados processados em uma tabela Delta:

```
python  
  
Copiar código  
  
from delta.tables import *  
  
# Ler os dados processados  
df = spark.read.csv("s3://meu-bucket/processed-data/", header=True,  
inferSchema=True)  
  
# Escrever no formato Delta  
df.write.format("delta").mode("overwrite").save("s3://meu-bucket/lakehouse-  
data/")
```

Mostrar como realizar consultas SQL diretamente no Delta:

```
python  
  
Copiar código  
  
spark.sql("CREATE TABLE lakehouse_table USING DELTA LOCATION  
's3://meu-bucket/lakehouse-data/'")  
  
result = spark.sql("SELECT * FROM lakehouse_table WHERE coluna =  
'valor'")  
  
result.show()
```

Consumindo os Dados

Passo 1 - Conectando o Delta Lake ao BI:

- Configurar uma instância do AWS Athena ou Spark Thrift Server para conectar com ferramentas como Power BI.
- Demonstrar consultas diretamente no BI.

Passo 2 - Relatórios e Dashboards:

- Criar relatórios em Power BI com gráficos sobre os dados.
- Explorar a interatividade do BI com filtros e agregações.

Data Warehouse (DW)

É um sistema de armazenamento de dados projetado para suportar análises complexas e relatórios. Ele organiza os dados de forma estruturada, usando esquemas como **estrela** ou **floco de neve**, com atenção em consistência, performance e integração de diferentes fontes.

Características:

- **Armazenamento Estruturado:** os dados são organizados em tabelas relacionais.
- **ETL (Extract, Transform, Load):** processos para extrair, transformar e carregar dados de diferentes sistemas.
- **Orientado a Análises:** otimizado para **consultas OLAP** (processamento analítico on-line).

Limitações:

- Não é ideal para dados não estruturados, como vídeos, imagens ou logs.
- Estrutura rígida dificulta a integração de novas fontes de dados.



Figura 1 – Data Warehouse

Fonte: <https://www.lanchuhuong.com/data-and-code/implementing-a-data-lake-or-data-warehouse-architecture-for-business-intelligence/> (2020)

Data Lake

É uma evolução que permite armazenar **dados estruturados, semi-estruturados e não estruturados** em seu formato bruto. Ele é construído para ser escalável e flexível, usando tecnologias de armazenamento distribuído como Amazon S3 ou Hadoop HDFS.

Características:

- **Armazenamento Heterogêneo:** pode armazenar desde planilhas até vídeos e dados IoT.
- **Economia de Custos:** usa arquitetura escalável para armazenamento de grandes volumes de dados.
- **Foco na Ingestão:** dados são carregados no formato bruto para processamento posterior.

Limitações:

- Falta de **transações ACID** (Atomicidade, Consistência, Isolamento, Durabilidade).

- Problemas com "Data Swamp" (acúmulo desorganizado de dados).

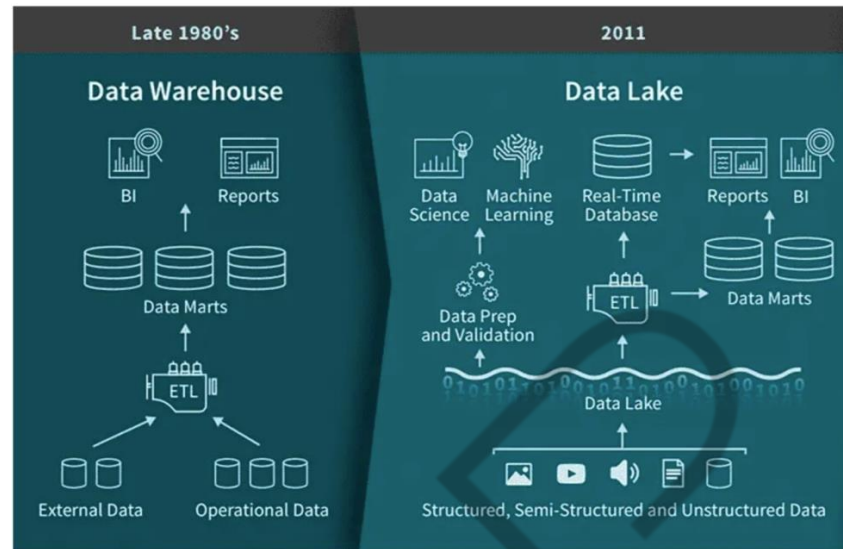


Figura 2 – Data Lake

Fonte: <https://www.lanchuhuong.com/data-and-code/implementing-a-data-lake-or-data-warehouse-architecture-for-business-intelligence/> (2020)

Data Lakehouse

Combina os benefícios do Data Lake (flexibilidade e suporte a dados não estruturados) com as funcionalidades transacionais e organizadas de um Data Warehouse.

Características:

- **Formato Delta (ou similar):** adiciona suporte a **transações ACID** no Data Lake.
- **Suporte a Consultas SQL:** permite explorar dados de forma estruturada diretamente do Data Lake.
- **Versionamento e Histórico:** facilita auditorias e rollback de dados.

Benefícios:

- Combina **analytics em tempo real** com **armazenamento econômico**.
- Elimina a necessidade de manter DWs e Lakes separados.

Principais Tecnologias:

- Apache Delta Lake

- Apache Iceberg
- Apache Hudi

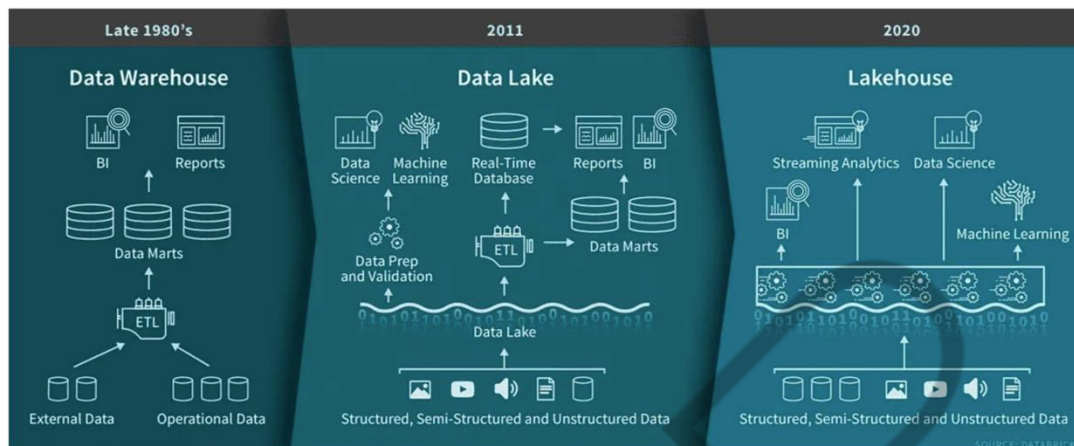


Figura 3 - Data Lakehouse

Fonte: <https://www.lanchuhuong.com/data-and-code/implementing-a-data-lake-or-data-warehouse-architecture-for-business-intelligence/> (2020)

O **ACID** é um conjunto de propriedades fundamentais para transações em sistemas de banco de dados, que garantem a integridade, confiabilidade e consistência dos dados. O acrônimo representa quatro princípios:

Atomicidade (Atomicity)

Tudo ou nada: uma transação deve ser concluída completamente ou não deve ser executada. Se qualquer parte falhar, todas as alterações feitas pela transação são revertidas.

Exemplo: ao transferir dinheiro entre contas, tanto o débito quanto o crédito precisam ser realizados; caso contrário, nenhum ocorre.

Consistência (Consistency)

Garante que uma transação leve o sistema de um estado válido para outro estado válido, obedecendo regras e restrições do banco de dados.

Exemplo: uma transação não pode violar regras como "o saldo de uma conta não pode ser negativo" ou "chaves primárias não podem ser duplicadas".

Isolamento (Isolation)

As transações simultâneas devem ser executadas de forma isolada, como se cada uma fosse a única em execução. Isso evita interferências entre elas.

Exemplo: duas transações de atualização em um mesmo registro não podem causar resultados incorretos devido a conflitos.

Durabilidade (Durability)

Depois de confirmada (commit), uma transação deve permanecer armazenada no sistema, mesmo em caso de falhas, como queda de energia ou desligamento inesperado.

Exemplo: uma transferência concluída entre contas deve estar persistida no banco de dados, mesmo após um desligamento do sistema.

Essas propriedades são amplamente utilizadas em bancos de dados relacionais (como MySQL, PostgreSQL) e tecnologias emergentes, como **Data Lakehouses**, para garantir confiabilidade em ambientes de dados críticos.

Data Fabric

Definição: é um conceito emergente de design de integração e gerenciamento de dados. Seu objetivo é dar suporte ao acesso de dados em todo o negócio por meio de integração de dados flexível, reutilizável, aumentada e, às vezes, automatizada.

Características:

- **Integração Unificada:** conecta dados de diferentes locais (on-premise, cloud, edge).
- **Automação Baseada em IA:** melhora a descoberta, qualidade e integração de dados.
- **Suporte Multi-Cloud:** permite orquestração em várias plataformas de nuvem.

Benefícios:

- Reduz complexidade operacional.
- Torna os dados acessíveis para análises e inteligência artificial.

Relação com Lakehouse:

- Um Data Fabric pode orquestrar o Data Lakehouse como um componente de sua arquitetura geral.

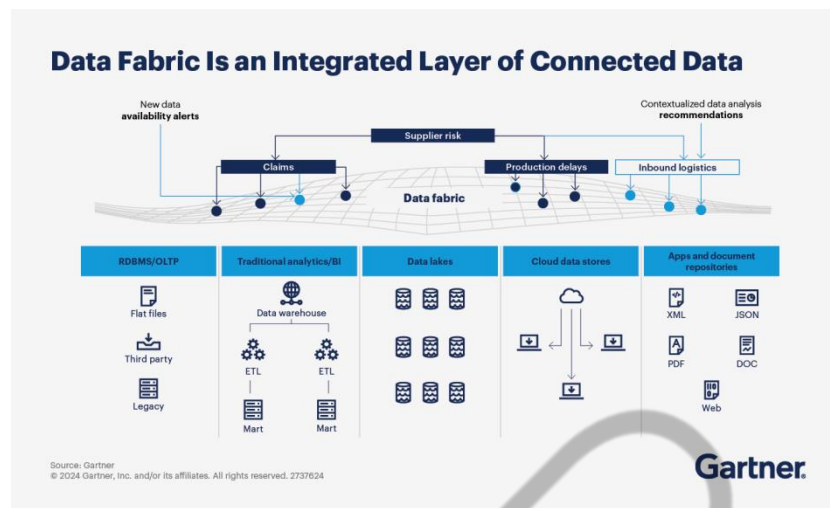


Figura 4 – Data Fabric
Fonte: [Gartner](#) (2025)

Data Mesh

O Data Mesh é um conceito descentralizado que propõe que os dados sejam tratados como **produtos** e que cada domínio (departamento ou equipe) seja responsável por seus próprios dados, garantindo escalabilidade e autonomia.

Princípios Fundamentais:

Domínios Autônomos: cada equipe gerencia seus próprios dados.

Dados como Produto: dados são tratados como produtos com APIs e contratos claros.

Plataforma de Autoatendimento: ferramentas para facilitar o uso e integração de dados.

Governança Federada: normas globais, mas adaptáveis por domínio.

Benefícios:

- Escalabilidade organizacional.
- Reduz dependência de equipes centrais de dados.
- Alinhamento direto com objetivos de negócios.

Relação com Data Lakehouse: um Data Mesh pode usar um Data Lakehouse como infraestrutura base para armazenar dados de domínios.

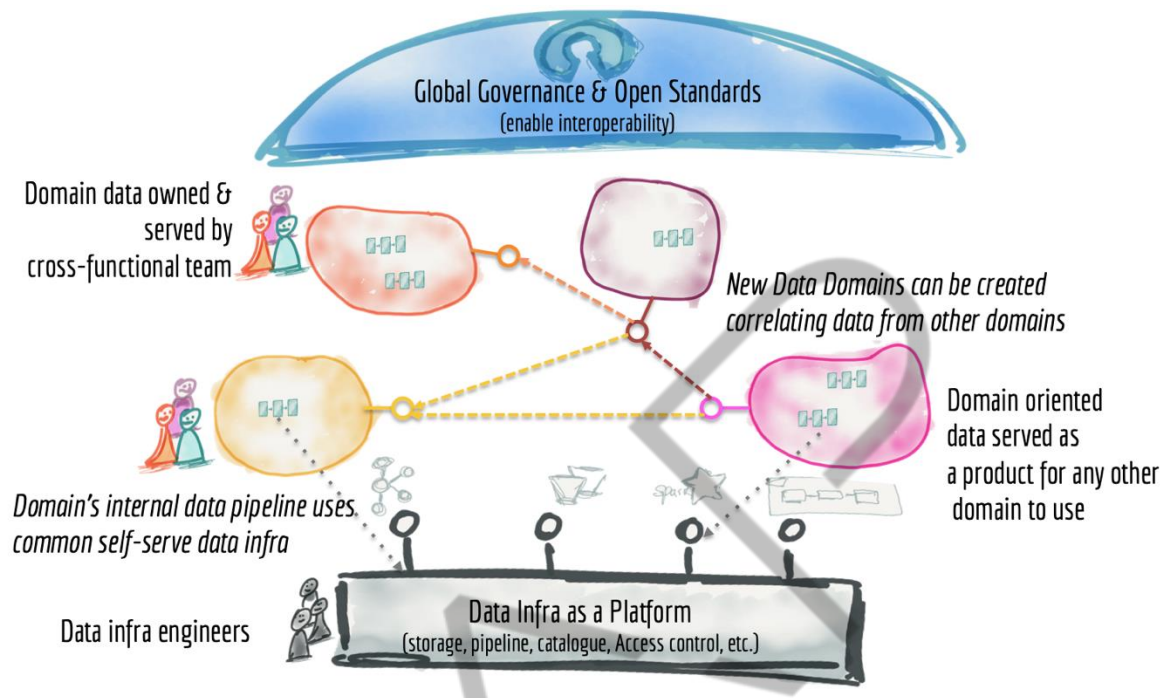


Figura 5 – Data Mesh
fonte: <https://martinfowler.com/articles/data-monolith-to-mesh.html> (2025)

Detalhes Técnicos dos Conceitos de Dados

Conceito	Data Warehouse	Data Lake	Data Lakehouse	Data Fabric	Data Mesh
Ano de Criação	1980s	2010s	2017	2020s	2019
Criador	Bill Inmon e Ralph Kimball	James Dixon (conceito popularizado com Hadoop)	Databricks	Gartner (conceito popularizado por IA e automação)	ZhamakDehghani (ThoughtWorks)
Uso	Análise estruturada de grandes volumes de dados	Armazenar dados brutos em formatos variados	Análise de dados estruturados e não estruturados com transações ACID	Orquestração de dados em ambientes híbridos e multi-cloud	Governança descentralizada e alinhamento de dados ao negócio

Principais Benefícios	Alta performance e consistência em análises	Flexibilidade e armazenamento econômico	Combinação de flexibilidade e governança com suporte a SQL	Integração unificada e suporte a IA	Escalabilidade e autonomia organizacional
Principais Fraquezas	Rigidez na ingestão de novos tipos de dados	Falta de governança e risco de Data Swamp	Complexidade na configuração inicial	Dependência de IA e ferramentas avançadas	Requer mudança cultural e técnica significativa

Tabela 1 – Conceito de dados
Fonte: elaborado pelo autor (2025)

MERCADO, CASES E TENDÊNCIAS

A evolução das arquiteturas de dados, como **Data Lakehouse**, **Data Fabric** e **Data Mesh**, tem transformado significativamente o mercado de gerenciamento de dados. Empresas de diversos setores estão adotando essas abordagens para melhorar a eficiência, escalabilidade e governança de seus dados.

Mercado

O mercado de gerenciamento de dados está em constante evolução, impulsionado pela necessidade de processar volumes crescentes de informações de maneira eficiente e segura. As arquiteturas modernas, como Data Lakehouse, Data Fabric e Data Mesh, oferecem soluções para os desafios enfrentados pelas empresas ao lidar com dados distribuídos e heterogêneos.

Cases de Sucesso

Empresas líderes em diversos setores têm implementado essas arquiteturas para otimizar suas operações e obter insights mais profundos a partir de seus dados.

Data Lakehouse: a [Databricks](#) desenvolveu uma plataforma de Lakehouse que combina a flexibilidade dos Data Lakes com as garantias transacionais dos Data Warehouses, permitindo que organizações realizem análises avançadas e Machine Learning em um único local.

Data Fabric: organizações têm adotado o Data Fabric para integrar dados de múltiplas fontes, facilitando o acesso e a análise em tempo real, o que resulta em decisões mais informadas e ágeis. [Veja](#) mais sobre o tema.

Data Mesh: [Empresas](#) que implementaram o Data Mesh conseguiram descentralizar a governança de dados, permitindo que diferentes equipes gerenciem seus próprios conjuntos de dados como produtos, aumentando a eficiência e a relevância dos insights obtidos.

Tendências

O futuro do gerenciamento de dados aponta para uma convergência dessas arquiteturas, onde as organizações buscam combinar os benefícios de cada abordagem para atender às suas necessidades específicas.

Integração de Arquiteturas: há uma tendência crescente de combinar elementos do Data Lakehouse, Data Fabric e Data Mesh para criar soluções híbridas que maximizem a flexibilidade, governança e escalabilidade dos dados.

Automação e IA: o uso de inteligência artificial e automação está se tornando central na gestão de dados, especialmente em arquiteturas de Data Fabric, para facilitar a integração e melhorar a qualidade dos dados.

Governança Descentralizada: o Data Mesh está promovendo uma mudança cultural nas organizações, incentivando a responsabilidade dos dados em nível de domínio e tratando-os como produtos, o que requer uma governança federada e colaborativa.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos as principais arquiteturas de gerenciamento de dados, desde os conceitos tradicionais de **Data Warehouse**, focados em análises estruturadas, até soluções modernas como **Data Lakehouse**, que unem flexibilidade e governança. Discutimos também as arquiteturas emergentes: **Data Fabric**, que conecta fontes de dados de forma integrada e automatizada, e **Data Mesh**, que descentraliza a governança e trata os dados como produtos.

Realizamos um hands-on com a criação de um **Data Lake** e sua evolução para um **Data Lakehouse**, utilizando ferramentas como Apache Spark e tabelas Delta, além de integrar essas arquiteturas em ferramentas de BI. Por fim, analisamos casos de sucesso e tendências do mercado, destacando a adoção de arquiteturas híbridas e o papel crescente da inteligência artificial e da automação na gestão de dados.

A aula forneceu uma visão prática e conceitual sobre o gerenciamento moderno de dados, preparando-os para entender os desafios e oportunidades no cenário atual.

REFERÊNCIAS

FOWLER, M. **How to Move Beyond a Monolithic Data Lake to a Distributed Data Mesh.** 2019. Disponível em: <https://martinfowler.com/articles/data-monolith-to-mesh.html>. Acesso em: 26 mar. 2025.

GARTNER. **How Data Fabric Can Optimize Data Delivery.**[s.d]. Disponível em: <https://www.gartner.com/en/data-analytics/topics/data-fabric>. Acesso em: 26 mar. 2025.

INMON, W. H. **Building the Data Warehouse.** Wiley, 1992.

KIMBALL, R., ROSS, M. **The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling.** Wiley, 2013.

SAWANT, A., SHAH, M. **Big Data Application Architecture Q&A: A Problem-Solution Approach.** Apress, 2014.

PALAVRAS-CHAVE

Data Warehouse. Data Lake. Data Lakehouse. Data Fabric. Data Mesh. Big Data. Machine Learning. Governança de Dados. Integração Multi-Cloud. Transações ACID. Dados como Produto.

EMENDAS

The background is a dark blue field filled with numerous small, light blue dots, resembling a starry sky. Overlaid on this are several large, wavy, translucent bands in shades of teal, blue, and yellow. These bands flow from the left side towards the right, creating a sense of motion. Scattered throughout the composition are various thin, light-colored geometric outlines: a circle with the number '7' inside, a small circle, an 'X' mark, a small circle, and a hexagon in the bottom right corner.

POSTECH